

DevOps 3 tier Project

CI/CD Implementation and Kubernetes

Deployment of Mock Exam Application

Name: Pooja Prakash Belgaumkar

- This project focuses on deploying a DevOps Mock Exam Application using a 3-tier architecture.
- The application is built, containerized, and deployed using Docker and Docker Compose, with CI/CD automation implemented through Jenkins.
- GitHub is used for source code management, and the project demonstrates basic DevOps practices such as automated build, scan, and deployment on an AWS EC2 instance.

1. First, launch an Ubuntu EC2 instance with 30 GB storage and connect to it using MobaXterm.

The screenshot displays the AWS Management Console interface for an EC2 instance. The top navigation bar shows the AWS logo, search, and navigation icons, along with the user's name 'PoojaPrakash' and the region 'Europe (Stockholm)'. The breadcrumb trail indicates the path: EC2 > Instances > i-0bd1f0313eb1032ab. The main content area is titled 'Instance summary for i-0bd1f0313eb1032ab (DevOps Exam App)'. It features a 'Connect' button, an 'Instance state' dropdown showing 'Running', and an 'Actions' dropdown. The instance was updated 5 minutes ago. The summary is organized into three columns: 1. General information: Instance ID (i-0bd1f0313eb1032ab), IPv6 address (none), Hostname type (IP name: ip-172-31-2-195.eu-north-1.compute.internal), Answer private resource DNS name (none), and Auto-assigned IP address (16.170.141.225 [Public IP]). 2. Network and state: Public IPv4 address (16.170.141.225), Instance state (Running), Private IP DNS name (IPv4 only) (ip-172-31-2-195.eu-north-1.compute.internal), Instance type (t3.micro), and VPC ID (vpc-04fbb5dfae5f66a87). 3. DNS and optimization: Private IPv4 addresses (172.31.2.195), Public DNS (ec2-16-170-141-225.eu-north-1.compute.amazonaws.com), Elastic IP addresses (none), and AWS Compute Optimizer finding (Opt-in to AWS Compute Optimizer for recommendations).

Instance summary for i-0bd1f0313eb1032ab (DevOps Exam App)		
Instance ID i-0bd1f0313eb1032ab	Public IPv4 address 16.170.141.225 open address	Private IPv4 addresses 172.31.2.195
IPv6 address -	Instance state Running	Public DNS ec2-16-170-141-225.eu-north-1.compute.amazonaws.com open address
Hostname type IP name: ip-172-31-2-195.eu-north-1.compute.internal	Private IP DNS name (IPv4 only) ip-172-31-2-195.eu-north-1.compute.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations.
Auto-assigned IP address 16.170.141.225 [Public IP]	VPC ID vpc-04fbb5dfae5f66a87	

2. Install Jenkins on the EC2 instance.

```
#!/bin/bash

# Install OpenJDK 17 JRE Headless
sudo apt install openjdk-17-jre-headless -y

# Download Jenkins GPG key
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key

# Add Jenkins repository to package manager sources
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

# Update package manager repositories
sudo apt-get update

# Install Jenkins
sudo apt-get install jenkins -y

~
~
~
~
~
~
~
```

```
root@ip-172-31-2-195:/home/ubuntu# java --version
openjdk 17.0.17 2025-10-21
OpenJDK Runtime Environment (build 17.0.17+10-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 17.0.17+10-Ubuntu-124.04, mixed mode, sharing)
root@ip-172-31-2-195:/home/ubuntu# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-01-26 08:39:00 UTC; 2min 27s ago
     Main PID: 12848 (java)
       Tasks: 39 (limit: 1008)
      Memory: 292.1M (peak: 336.9M)
         CPU: 1min 18.179s
       CGroup: /system.slice/jenkins.service
              └─12848 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins-

Jan 26 08:38:07 ip-172-31-2-195 jenkins[12848]: 0f3689b72bde4c0dafc2bb0fac75f39a
Jan 26 08:38:07 ip-172-31-2-195 jenkins[12848]: This may also be found at: /var/lib/jen
Jan 26 08:38:07 ip-172-31-2-195 jenkins[12848]: *****
Jan 26 08:38:07 ip-172-31-2-195 jenkins[12848]: *****
Jan 26 08:38:07 ip-172-31-2-195 jenkins[12848]: *****
Jan 26 08:39:00 ip-172-31-2-195 jenkins[12848]: 2026-01-26 08:39:00.644+0000 [id=31]
Jan 26 08:39:00 ip-172-31-2-195 jenkins[12848]: 2026-01-26 08:39:00.665+0000 [id=24]
Jan 26 08:39:00 ip-172-31-2-195 systemd[1]: Started jenkins.service - Jenkins Continuou
Jan 26 08:39:01 ip-172-31-2-195 jenkins[12848]: 2026-01-26 08:39:01.433+0000 [id=49]
Jan 26 08:39:01 ip-172-31-2-195 jenkins[12848]: 2026-01-26 08:39:01.434+0000 [id=49]
root@ip-172-31-2-195:/home/ubuntu#
```

3. Install Docker on the EC2 instance, add the Jenkins user to the Docker group, and restart Jenkins to apply the changes.
4. Verify that Jenkins can access Docker.

```
#!/bin/bash

# Update package manager repositories
sudo apt-get update

# Install necessary dependencies
sudo apt-get install -y ca-certificates curl

# Create directory for Docker GPG key
sudo install -m 0755 -d /etc/apt/keyrings

# Download Docker's GPG key
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc

# Ensure proper permissions for the key
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add Docker repository to Apt sources
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Update package manager repositories
sudo apt-get update

sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

```
root@ip-172-31-2-195:/home/ubuntu# sudo usermod -aG docker jenkins
root@ip-172-31-2-195:/home/ubuntu# sudo systemctl restart jenkins
root@ip-172-31-2-195:/home/ubuntu# sudo -u jenkins docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
root@ip-172-31-2-195:/home/ubuntu# sudo apt-get update
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Ign:5 https://pkg.jenkins.io/debian-stable binary/ InRelease
Hit:6 https://download.docker.com/linux/ubuntu noble InRelease
Get:7 https://pkg.jenkins.io/debian-stable binary/ Release [2044 B]
Get:8 https://pkg.jenkins.io/debian-stable binary/ Release.gpg [833 B]
Ign:8 https://pkg.jenkins.io/debian-stable binary/ Release.gpg
Reading package lists... Done
```

5. Configure Docker Hub credentials in Jenkins for pushing Docker images.

```
root@ip-172-31-2-195:/home/ubuntu# docker login -u poojaprakash08

Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit https://app.docker.com/settings

Password:

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-2-195:/home/ubuntu#
```

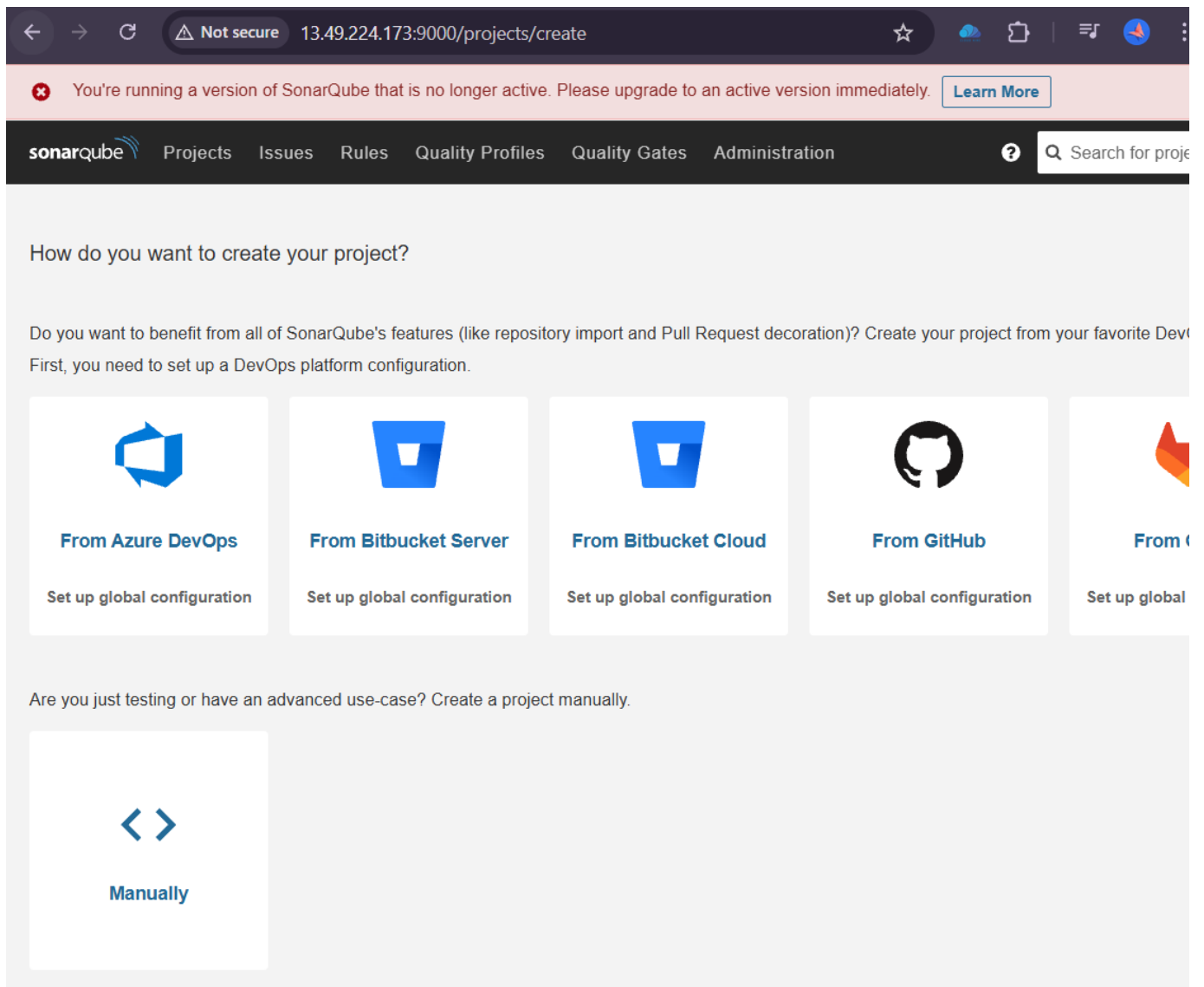
```
root@ip-172-31-2-195:/home/ubuntu# curl -sSfL https://raw.githubusercontent.com/docker/scout-cli/main/install.sh | sh -s -- -b /usr/local/bin
[info] fetching release script for tag='v1.19.0'
[info] using release tag='v1.19.0' version='1.19.0' os='linux' arch='amd64'
[info] installed /usr/local/bin/docker-scout
root@ip-172-31-2-195:/home/ubuntu# sudo chmod 777 /var/run/docker.sock
root@ip-172-31-2-195:/home/ubuntu#
```

6. Install Docker Compose, Trivy for security scanning, and SonarQube for code quality analysis.

```
root@ip-172-31-2-195:/home/ubuntu# docker run -d --name sonar -p 9000:9000 sonarqube:lts-community
Unable to find image 'sonarqube:lts-community' locally
lts-community: Pulling from library/sonarqube
e24a8b9e652f: Pull complete
5d5a1fad7028: Pull complete
eb27e3a98da1: Pull complete
4f4fb700ef54: Pull complete
c7ad1fe61e07: Pull complete
60d98d907669: Pull complete
f3929ce9ef98: Pull complete
1df735f481ad: Pull complete
ce2bd68be4fc: Download complete
600f318704dc: Download complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts-community
22641c35208fd5cba6e8ab05427a66bb164e14bf294a574f467f9523198ab466
root@ip-172-31-2-195:/home/ubuntu# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	Info → EXTRA	In Use
hello-world:latest	05813aedc15f	25.9kB	9.52kB		
sonarqube:lts-community	f709975ab31d	1.02GB	398MB	U	

```
root@ip-172-31-2-195:/home/ubuntu# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED   STATUS    PORTS    NAMES
root@ip-172-31-2-195:/home/ubuntu#
```



7. Install the required plugins in Jenkins.

8. Configure SonarQube in Jenkins for static code analysis.

Dashboard > Manage Jenkins > Tools

SonarQube Scanner installations

Add SonarQube Scanner

☰ SonarQube Scanner

×

Name

sonar-scanner

☒ Install automatically ?

☰ Install from Maven Central

×

Version

SonarQube Scanner 8.0.1.6346

▼

Add Installer ▼

Docker installations

Add Docker

☰ Docker

×

Name

docker

☒ Install automatically ?

☰ Download from docker.com

×

Docker version ?

latest

Add Installer ▼

Add Docker

Save

Apply

9. Configure Docker Hub credentials in Jenkins for pushing Docker images.

Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted) >

New credentials

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

poojaprakash08

☐ Treat username as secret ?

Password ?

.....

ID ?

docker|

Create

10. Configured Jenkins in SonarQube and added the SonarQube authentication token as credentials in Jenkins so that both tools can communicate securely for code analysis.

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

Search for projects...

Tokens of Administrator

Generate Tokens

Name Expires in

Enter Token Name 30 days Generate

New token "token" has been created. Make sure you copy it now, you won't be able to see it again!

Copy squ_8d1d6e9d9129a04b091f5279bb920c44807895a4

Name	Type	Project	Last use	Created	Expiration	
token	User		Never	January 26, 2026	February 25, 2026	Revoke

Done

Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted) >

New credentials

Kind

Secret text

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

.....

ID ?

sonar-token

Description ?

sonar-token

Create

11. Now we have both Docker and SonarQube credentials configured in Jenkins to communicate with them.

Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted) >

Global credentials (unrestricted)

+ Add Credentials

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
 docker	poojaparakash08/*****	Username with password	
 sonar-token	sonar-token	Secret text	sonar-token 

Icon: S M **L**

12. A webhook is created in SonarQube to notify Jenkins about the code analysis result and automatically trigger the next pipeline actions after quality check.

Create Webhook

All fields marked with * are required

Name *

jenkins ✓

URL *

http://13.49.224.173:8080/sonarqube-webhook/ ✓

Server endpoint that will receive the webhook payload, for example:
"http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example:
"https://myLogin:myPassword@my_server/foo"

Secret

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

[Create](#) [Cancel](#)

13. Create a Jenkins pipeline job using pipeline syntax.

Dashboard > All > New Item

New Item

Enter an item name

DevOpsExamApp

Select an item type



Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.



Pipeline
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a

OK

14. Build the Jenkins job to execute all pipeline stages.

Dashboard > DevOpsExamApp >

Status

</> Changes

Build Now

Configure

Delete Pipeline

Full Stage View

Rename

Pipeline Syntax

Builds

Filter

Today

#5 12:46 PM

✓ DevOpsExamApp

Add description

Last Successful Artifacts

trivy-fs-report.html 16.87 KiB view

Stage View

Average stage times:
(full run time: ~59s)

	Git Checkout	File System Scan	Verify Docker Compose	Build Docker Image	Deploy with Docker Compose	Verify Deployment	Declarative: Post Actions
#5	643ms	3s	597ms	10s	38s	1s	2s

Permalinks

- Last build (#5), 13 min ago
- Last stable build (#5), 13 min ago
- Last successful build (#5), 13 min ago
- Last completed build (#5), 13 min ago

15. Finally, access the DevOps Mock Exam application using the public IP address with port 5000.

The screenshot shows a web browser window with the address bar displaying "Not secure" and the IP address "13.49.224.173:5000". The page features a purple header with the text "Kastro's Mock Exam". Below this, the main heading is "DevOps Mock Certification Exam" in a large, bold, blue font. Underneath the heading, a subtitle reads "Validate your expertise with of DevOps learning". To the left of the main content area is a light purple circle. To the right, there are three white buttons with purple icons and text: "Timed Assessment", "Real-world Scenarios", and "Performance Analytics". In the center, there is a white form box with a purple border. The form contains three fields: "Full Name" with the value "Kastro Kiran V", "Gender" with a dropdown menu showing "Select Gender", and "Email Address" with the value "your@email.com".

The screenshot shows a web browser window with the address bar displaying "Not secure" and the URL "13.49.224.173:5000/start". The page features a purple header with the text "Kastro's Mock Exam". Below this, the main heading is "DevOps Professional Certification Exam" in a large, bold, blue font. Underneath the heading, a subtitle reads "Validate your DevOps knowledge and skills". In the center, there is a purple button with a white clock icon and the text "14:57". Below this, there is a white box containing three fields: "Candidate Name: Pooja", "Email: poojaprakash1808@gmail.com", and "Gender: Female". Below the white box, there is a question: "1 What is the primary purpose of Jenkins in a CI/CD pipeline?". There are four radio button options: "Container orchestration", "Automating builds, tests, and deployments", "Static code analysis", and "Artifact repository management".